

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING**

Khwopa College Of Engineering
Libali, Bhaktapur
Department of Computer Engineering



**A PROPOSAL ON
FACIAL RECOGNITION SYSTEM ALONG WITH
REAL TIME TRACKING**

Submitted in partial fulfillment of the requirements for the degree

BACHELOR OF COMPUTER ENGINEERING

Submitted by

Gobinda Bista

Hari Krishna Khuju

Subash Bhandari

Suresh Shrestha

KCE077BCT015

KCE077BCT016

KCE077BCT037

KCE077BCT040

Under the Supervision of

Er.Sushil Dyopala

Department Of Computer Engineering

Khwopa College Of Engineering
Libali, Bhaktapur

Abstract

Facial recognition technology has witnessed remarkable advancements in recent years, emerging as a pivotal tool across diverse industries for identity verification and access control. The proposed system aims to enhance security measures, streamline access control, and provide valuable insights into user behaviors across various applications. This proposal introduces a comprehensive system that integrates multi-target facial recognition with real-time tracking capabilities. Leveraging cutting-edge computer vision algorithms, deep learning-based face recognition algorithms and real-time tracking software, this system offers a versatile solution applicable in settings ranging from secure facilities to retail environments. The system architecture incorporates facial recognition algorithms, a database of facial templates, high-quality camera systems, and real-time tracking software. By providing instant identification and continuous tracking of recognized individuals, the system offers a robust security mechanism and the ability to monitor user activities in real-time.

Keywords: Face detection, Face recognition, Tracking multiple face

Contents

Abstract	2
Contents	2
List of Figures	4
List of Symbols and Abbreviation	6
1 Introduction	1
1.1 Background Introduction	1
1.2 Problem Statement	2
1.3 Objective	2
2 Literature Review	3
3 FEASIBILITY STUDY	5
3.0.1 Technical Feasibility	5
3.0.1.1 System Architecture	5
3.0.1.2 Technology Stack	5
3.0.1.3 Algorithm Selection	5
3.0.1.4 Integration with Existing Systems	5
3.0.1.5 Scalability	5
3.0.1.6 Data Security and Privacy	6
3.0.2 Operational Feasibility	6
3.0.2.1 User Training	6
3.0.2.2 User Acceptance Testing (UAT)	6
3.0.2.3 Operational Impact	6
3.0.2.4 Maintenance and Support	6
3.0.3 Economical Feasibility	6
3.0.3.1 Cost Estimation	6
3.0.3.2 Return on Investment (ROI)	6
3.0.3.3 Funding Sources	6
3.0.3.4 Risk Assessment	6
4 Methodology	7
4.1 Software Development Methodology	7
4.1.1 Agile Methodology	7
4.2 ALGORITHMS AND MODELS	8
4.2.1 Haar Cascades	8
4.2.2 Local Binary Patterns	9
4.2.3 FaceNet	9
4.2.4 DeepID	9

4.2.5	Fisherfaces	9
4.2.6	Convolutional Neural Networks	9
4.2.7	MTCNN (Multi-task Cascaded Convolutional Networks) . .	9
4.2.8	KLT Tracker (Kanade-Lucas-Tomasi Tracker)	10
4.2.9	Multiple Object Tracking Algorithms(MOT)	10
4.2.10	CSRT (Discriminative Correlation Filter with Channel and Spatial Reliability)	10
4.2.11	Tracking Algorithm	10
4.3	Proposed System	11
4.3.1	Dataset Creation and Image Preprocessing	11
4.3.2	Face Detection	11
4.3.3	Face Recognition and Calculation	12
4.3.4	DataBase	12
4.3.5	Real-Time Tracking	12
5	Implementation Plan	14
5.1	Schedule(Gantt-Chart)	14
5.2	SOFTWARE REQUIREMENT	14
5.2.1	Python	14
5.2.2	OpenCV (Open Source Computer Vision Library)	14
5.2.3	MySQL	15
5.2.4	Tensor Flow	15
5.3	FUNCTIONAL REQUIREMENT	15
5.3.1	Face Detection	15
5.3.2	Face Recognition	15
5.3.3	Database Management	15
5.3.4	Real-time Monitoring	15
5.4	NON-FUNCTIONAL REQUIREMENT	16
5.4.1	Performance	16
5.4.1.1	Response Time	16
5.4.1.2	Throughput	16
5.4.2	Scalabiity	16
5.4.3	Reliability	16
5.4.4	Availability	16
5.4.5	Security	16
5.4.5.1	Data Security	16
5.4.5.2	Access Control	16
6	Expected Outcomes	17
6.1	Conclusion	18
	Bibliography	19

List of Figures

4.1	System Block Diagram	11
4.2	System architecture of face recognition system	12
4.3	FaceNet Architecture	13
4.4	Triplet Loss	13
5.1	Gantt Chart	14
6.1	face recognition output	17
6.2	multiPerson-Tracking	18

List of Symbols and Abbreviation

CSRT	Discriminative Correlation Filter with Channel and Spatial Reliability
DeepSHORT	Deep Simple Online and Real-Time Tracking
FTNet	Fast Track Net
GUI	Graphical User Interface
HD	High Definition
KLT	Kanade Lucas Tomasi Tracker
LDA	Linear Discriminant Analysis
LBP	Local Binary Patterns
LFA	Local Features Analysis
MOT	Multi Object Tracking
MMFNet	Min Mobile FaceNet
MTCNN	Multi-Task Cascaded Convolution Neural Network
OpenCV	Open Source Computer Vision Library
PCA	Prompt Corrective Action
PHP	Hypertext Preprocessor
RDBMS	Relational Database Management System
SQL	Structured Query Language
SVM	Support Vector Machine
TCO	Total Cost of Ownership
UAT	User Acceptance Testing
YOLO-v3	You Only Look Once Version3

Chapter 1

Introduction

1.1 Background Introduction

In an era marked by rapid technological advancement, the fusion of facial recognition and real-time tracking stands at the forefront of transformative innovations. Facial recognition technology, with its roots in computer vision and artificial intelligence, has evolved into a powerful means of biometric identification. Concurrently, real-time tracking adds a dynamic dimension to this capability, enabling continuous monitoring and analysis of individuals' movements. This fusion of facial recognition and real-time tracking holds immense potential across a spectrum of industries, reshaping security protocols, refining access control mechanisms, and revolutionizing user engagement experiences.

Facial recognition technology has its roots in the 1960s, with initial experiments focused on pattern recognition in still images. Early developments primarily revolved around manual feature extraction and matching. However, it wasn't until the advent of machine learning and deep neural networks that facial recognition achieved unprecedented accuracy and efficiency. Contemporary systems leverage convolutional neural networks (CNNs) and other advanced algorithms to extract intricate facial features, allowing for highly reliable identification.

Facial recognition technology has diverse applications. In security and law enforcement, it aids in criminal identification and public safety. In consumer electronics, it facilitates secure access to devices. The retail sector employs facial recognition for personalized marketing and customer experiences. As the technology matures, its relevance extends to education, healthcare, and various facets of daily life.

Real-time tracking addresses the limitations of static facial recognition by introducing continuous monitoring capabilities. This evolution allows systems to track individuals dynamically, adapting to their movements in real-time. Real-time tracking has found applications in security, retail analytics, and personalized user experiences, providing a more holistic and context-aware solution.

Facial recognition systems often leverage deep learning techniques, while real-time tracking incorporates sophisticated algorithms for object detection and motion analysis. The integration of these technologies is pushing the boundaries of what is possible in terms of security, access control, and user engagement. This proposal will explore the intricacies of combining facial recognition with real-time tracking, shedding light on the technical, ethical, and practical considerations that accompany this integration.

1.2 Problem Statement

The rapid evolution of facial recognition technology, while transformative, presents a series of challenges and limitations that hinder its full potential. Despite achieving remarkable accuracy in static identification, traditional facial recognition systems lack the agility to address dynamic scenarios and real-time changes in individuals' positions. This inadequacy becomes particularly evident in security contexts, access control systems, and user engagement applications where continuous monitoring and adaptability are paramount.

Challenges in Traditional Facial Recognition:

Static Nature:

Traditional facial recognition systems operate on the assumption of a static environment, making them less effective in scenarios where individuals are in motion. This limitation compromises the system's accuracy and responsiveness, especially in high-traffic areas or situations requiring immediate action.

Limited Context Awareness:

Facial recognition, when divorced from real-time tracking, lacks the contextual awareness needed to respond dynamically to individuals' movements. The inability to adapt to changing contexts diminishes the system's efficacy in applications requiring continuous monitoring and analysis.

Scalability Challenges:

Large-scale deployments face scalability challenges when dealing with a high volume of individuals in crowded spaces. Performance degradation and increased false positives/negatives can compromise the overall effectiveness of facial recognition systems.

Need for Continuous Monitoring:

Security threats are not static; they evolve over time and often involve individuals in motion. A static facial recognition approach may fail to detect or respond adequately to security threats that manifest in dynamic scenarios.

Access control systems demand real-time responsiveness to grant or deny entry based on dynamic factors. Traditional facial recognition's inability to adapt in real-time compromises the integrity of access control mechanisms.

1.3 Objective

The aforementioned challenges underscore the need for an integrated system that combines facial recognition with real-time tracking. Such an integration seeks to address the static limitations of traditional facial recognition, enhance contextual awareness, and provide continuous monitoring capabilities. By doing so, the proposed system aims to overcome the identified challenges, offering a more versatile, adaptive, and ethically sound solution for applications ranging from security to user engagement.

Chapter 2

Literature Review

In this paper, authors have used deep learning methods for the fiducial points extraction and embedding. The best results were obtained using a customized architecture of residual network with 18 layers, drastically reducing the number of parameters from 19M to 700k. Support Vector Machine (SVM) is used for classification task since it is fast for both training and inference. The system achieved an error rate of 0.12103 for facial features detection. This system had competitive accuracy (0.95 ± 0.13), runs in real-time (23.23 ± 0.60 fps) [6].

In order to achieve real-time detection performance on embedded systems with limited computing power, this paper used one stage detector YOLOv3 algorithm to achieve face recognition, and some experimental optimizations are made for the YOLOv3 algorithm depend on the task. They optimized the computational efficiency using TensorRT. TensorRT is a C++ library from NVIDIA; it is used for high-performance inference on NVIDIA GPUs and deep learning accelerators. They used an Nvidia GTX 1080 Ti GPU to train the model through the Darknet framework [31] in an Ubuntu 18.04 environment. During training, They used the SGD with a batch size of 16, the momentum and weight decay are respectively set as 0.95 and 0.005, and adopt batch normalization [3].

This paper proposes a multi-target real-time detection, tracking, and recognition algorithm, which includes three strategies: fast detection, fast-tracking, and quick recognition. First, the human faces are detected by a rapid detection strategy, and MMFNet is used to extract features quickly. Finally, fast face tracking through FTNet. The algorithm avoids the repeated detection and recognition of the same target, and the tracking is stable. It successfully converts the multi-target scene into a single target scene. FTNet achieved high tracking accuracy on OTB and 300VW, and the tracking speed reached 102fps on the Xiph and camera data test set, which improved the tracking speed while ensuring high tracking accuracy. Although MMFNet has reduced efficiency in face recognition, the model size reduced by 45%, and the recognition speed has increased by 25%. The experimental results indicate that the strategy proposed in this paper balances out the problem of time-consuming multi-target tracking and detection in multi-target scenarios. It has sufficient accuracy and real-time performance in detecting and identifying human faces in multi-target scenes [2].

In this thesis a system was proposed and implemented to perform real time facial detection and recognition using a drone, in conjunction with a computer, to find and track a specified target individual. The facial recognition systems

were found to be the key component of the entire system, so three different facial recognition systems were implemented to quantify their performance. All though, the different facial recognition systems had varying strengths and weaknesses, FaceNet performed best when the system was tested in a real-world setting. The results of this thesis indicate that using facial recognition algorithms in conjunction with a drone with the goal of tracking an individual is possible. Even when using low cost, light weight drones, facial recognition is possible by having the drone stream video to a computer powerful enough to handle facial recognition. By streaming video to a computer, this system solved many of the performance issues mentioned in papers that implemented similar systems [2;7]. There exist extraneous factors which limit the capabilities of this system; but it served as a strong foundation that can be built upon [5].

In this paper, we implement a real-time surveillance consist of face recognition and invader tracking which based on OpenCV. OpenCV provides a set of open source programming libraries which suitable for real-time surveillance system. In this paper, we propose to use AdaBoost Cascade combined with skin model for face detection and then in order to recognize the candidate faces, they will be analyzed by the hybrid Wavelet and LDA method. After that, Mean shift and Kalman filter will be invoked to track the face. The user interface provides the current state, camera view and records for cross-platform user. The experimental results show that the algorithm has quite good performance in terms of real-time and accuracy [4].

Biometric systems are structures that have been often used in recent years. This biometric system is based on the use of some physiological features of the person for security. It is predicted that biometric systems will be an indispensable part of the information security systems in the coming years. Thus, people won't have to carry cards and/or memorize their own passwords. In this study, a biometric system for human face detection and recognition is put into practice. This system put into practice works real time and successfully carries out face recognition, detection and tracking. In this way, the system has the feature of being integrated into different practices. In this study, face detection and tracking system, is put into practice in a personal automation. Here, the aim is to carry out personnel's entry and exit automatically and safely. In this system practiced basing on PCA, in the future studies, various optimization algorithms can be used in order to increase system performance [1].

It has been certainly made out from the result and analysis that the proposed work has provided a better implementation of the current algorithms with integration of the modified work. The proposed work of tracking people in live video is just small part of the much larger picture. Compared to legacy systems, assay of facial expressions can help expedite a lot of manual labor that goes into the respective area of work, unnecessarily. Similar approach can be used to further develop system which detect crimes, identify mood of people, security systems, etc. Live Subject tracking, Facial Recognition and Analysis is a much deeper subject which provides immense research opportunities and also development for the greater good of the society [7].

Chapter 3

FEASIBILITY STUDY

A feasibility study for a Facial Recognition Along With Real-Time Tracking is crucial to assess the viability of implementing such a system within an organization. The study typically examines various aspects to determine if the proposed solution is feasible from technical, economic, operational, and scheduling perspectives. Here are key components of a feasibility study for a Facial Recognition Along With Real-Time Tracking:

3.0.1 Technical Feasibility

3.0.1.1 System Architecture

Define the technical architecture of the real-time face tracking system. Specify the hardware and software components required for implementation.

3.0.1.2 Technology Stack

Identify the technologies, frameworks, and tools needed for real-time face tracking. Ensure compatibility with existing systems if any.

3.0.1.3 Algorithm Selection

Choose appropriate face tracking algorithms (e.g., KLT, MOSSE, deep learning-based methods). Assess the computational requirements and efficiency of the chosen algorithms.

3.0.1.4 Integration with Existing Systems

Evaluate how the system integrates with other existing systems or databases. Assess compatibility and potential challenges.

3.0.1.5 Scalability

Assess the scalability of the system to handle varying loads. Consider the ability to scale horizontally or vertically based on demand.

3.0.1.6 Data Security and Privacy

Implement measures to ensure the security and privacy of facial data. Address encryption, access controls, and compliance with privacy regulations.

3.0.2 Operational Feasibility

3.0.2.1 User Training

Develop a training plan for end-users to interact with the face tracking system. Ensure ease of use and user-friendly interfaces.

3.0.2.2 User Acceptance Testing (UAT)

Conduct UAT to gather feedback from end-users. Identify and address any usability issues or concerns.

3.0.2.3 Operational Impact

Assess how the face tracking system will integrate into existing operations. Minimize disruptions and downtime during the implementation phase.

3.0.2.4 Maintenance and Support

Establish a plan for ongoing system maintenance and support. Ensure that the system remains operational with minimal downtime.

3.0.3 Economical Feasibility

3.0.3.1 Cost Estimation

Provide a detailed breakdown of costs associated with developing and implementing the face tracking system. Include hardware, software, development, deployment, and maintenance costs.

3.0.3.2 Return on Investment (ROI)

Estimate the potential ROI based on anticipated benefits. Consider time savings, improved security, or other relevant factors.

3.0.3.3 Funding Sources

Identify potential funding sources for the project. Explore internal budgets, external grants, or partnerships. Total Cost of Ownership (TCO): Calculate the TCO over the expected lifespan of the system. Include operational, maintenance, and upgrade costs.

3.0.3.4 Risk Assessment

Identify potential risks that could impact the economic feasibility. Develop mitigation strategies for high-impact risks.

Chapter 4

Methodology

4.1 Software Development Methodology

An iterative and incremental software development approach, such as Agile methodology, can be adopted. Agile methodologies emphasize flexibility, collaboration, and iterative development, which align well with the nature of the project and its potential changes and refinements.

4.1.1 Agile Methodology

Agile development is a flexible and iterative approach to software development that prioritizes adaptability and collaboration. It emphasizes delivering small, incremental releases of a product with the goal of responding quickly to changing requirements and feedback. Agile methodologies are based on the Agile Manifesto, which values individuals and interactions, working solutions, customer collaboration, and responding to change over rigid processes and tools. The Agile development process is typically organized into iterations or sprints, each of which includes the following key steps:

- **Requirements:** In Agile, requirements are expressed as user stories, which are short descriptions of a feature told from the perspective of the end user. Typically, the product owner collaborates with stakeholders to define and prioritize user stories, which in our case, are the objectives and highlights of the problem statement. These stories are added to the product backlog.
- **Design:** Design involves planning how the requirements will be implemented, considering the overall architecture, user interface, and other relevant aspects. During sprint planning, the development team discusses and plans how to address the problem statements and achieve the objectives. Design decisions are made collaboratively, and the team may create mock-ups or prototypes to visualize the solution.
- **Development:** This phase involves coding or implementing the solutions outlined in the design phase. Development work is carried out based on the

tasks assigned to team members during the sprint planning. Continuous collaboration and communication are crucial to address any issues or changes that may arise.

- **Testing:** Testing is an integral part of Agile development and involves validating that the implemented features meet the specified requirements. Testing will be done parallelly with development, creating test cases based on user stories and conducting various testing types (unit testing, integration testing, etc.) to ensure the quality of the code and output.
- **Deploy:** Deployment involves making the developed features available for use by end-users. The product increment is deployed to a staging or production environment. Continuous integration and deployment practices ensure that the latest changes are automatically integrated and deployed when ready.
- **Review:** At the end of each iteration, a review or demo is conducted to showcase the completed features to stakeholders. The team will participate in a review meeting to provide feedback, discuss any adjustments needed, and reprioritize the product backlog.
- **Launch:** The final step involves releasing the product increment to users. In our project, a web interface will be rolled at launch which will facilitate user interaction. Once all necessary adjustments are made based on feedback and the priorities based on objectives the product is officially launched. The cycle then repeats with a new iteration.

It's important to note that the design, development, testing, deployment, review, and launch steps occur iteratively and multiple times throughout the project. Agile development promotes continuous improvement and adaptation to changing requirements, allowing teams to deliver value to users more frequently and efficiently.

4.2 ALGORITHMS AND MODELS

Facial recognition systems rely on various algorithms and models to accurately identify and verify individuals based on facial features. Here are some common algorithms and models used in facial recognition systems:

4.2.1 Haar Cascades

1. **Description:** Haar Cascades is an object detection algorithm used for identifying faces in images or video streams with accuracy of 80% - 95%. It works by training on positive and negative images to create a classifier.

2. Application: Initial face detection before more detailed analysis.

4.2.2 Local Binary Patterns

1. Description: LBP is a texture descriptor that is often used for facial feature extraction. It focuses on local patterns of pixel intensities.
2. Application: Feature extraction in facial recognition systems.

4.2.3 FaceNet

1. Description: FaceNet is a deep learning model for face recognition that maps faces into a high-dimensional embedding space having accuracy of 99%.
2. Key Features: It uses a triplet loss function during training to ensure that embeddings of the same person are close together, while embeddings of different people are far apart.

4.2.4 DeepID

1. Description: Utilizes deep convolutional neural networks to learn discriminative features for face identification.
2. Application: Face recognition with a focus on handling variations in pose, lighting, and expression.

4.2.5 Fisherfaces

1. Description: A variation of eigenfaces that uses Linear Discriminant Analysis (LDA) to maximize class separability.
2. Application: Enhanced face recognition performance.

4.2.6 Convolutional Neural Networks

1. Description: CNNs are deep learning models that automatically learn hierarchical features from raw pixel data with accuracy of 95% - 99%. They excel at capturing spatial hierarchies in images.
2. Application: State-of-the-art facial recognition, especially when dealing with large datasets and complex variations.

4.2.7 MTCNN (Multi-task Cascaded Convolutional Networks)

1. Description: Designed for real-time face detection and alignment, often used as an initial step in facial recognition systems with accuracy exceeding 95%.
2. Application: Initial face detection before recognition.

4.2.8 KLT Tracker (Kanade-Lucas-Tomasi Tracker)

1. Description: Feature-based tracking which utilizes corner points and features in an image to track objects by estimating their motion. Effective for tracking objects with consistent features. It is sensitive to variations in lighting and occlusions.
2. Application: Generally fast and suitable for real-time tracking.

4.2.9 Multiple Object Tracking Algorithms(MOT)

1. Description: MOT algorithms aim to track multiple objects simultaneously in a video sequence. Often involve a combination of detection, tracking, and association methods. Accuracy can be impacted by crowded scenes, similar appearance of objects, and variations in object size. Depending on the algorithm, real-time performance is achievable. Computational complexity may increase with the number of tracked objects.
2. Application: Capable of handling scenarios with multiple objects and occlusions. Accuracy can be impacted by crowded scenes, similar appearance of objects, and variations in object size.

4.2.10 CSRT (Discriminative Correlation Filter with Channel and Spatial Reliability)

1. Description: Integrates discriminative correlation filters with spatial and channel reliability to improve object tracking.
2. Application: Real-time object tracking with improved accuracy.

4.2.11 Tracking Algorithm

1. **kalman filter**
Kalman Filter is effective for tracking objects with linear motion and predictable trajectories. Computationally efficient and suitable for real-time applications. Can handle noisy measurements and predictions.
2. **Correlation Filter Tracking**
it's computationally efficient and suitable for real-time tracking. Can handle scale variations, rotations, and partial occlusions. Effective for short-term tracking scenarios.
3. **DeepSORT(Deep Simple Online and Real-Time Tracking)**
IT integrates deep features for improved tracking accuracy. Effective in maintaining consistent track identities across frames. Suitable for scenarios with occlusions and appearance changes.

4.3 Proposed System

This system will be able to detect student or employee without physical contact and update the database based on the response from system. The system architecture of the proposed system is given below,

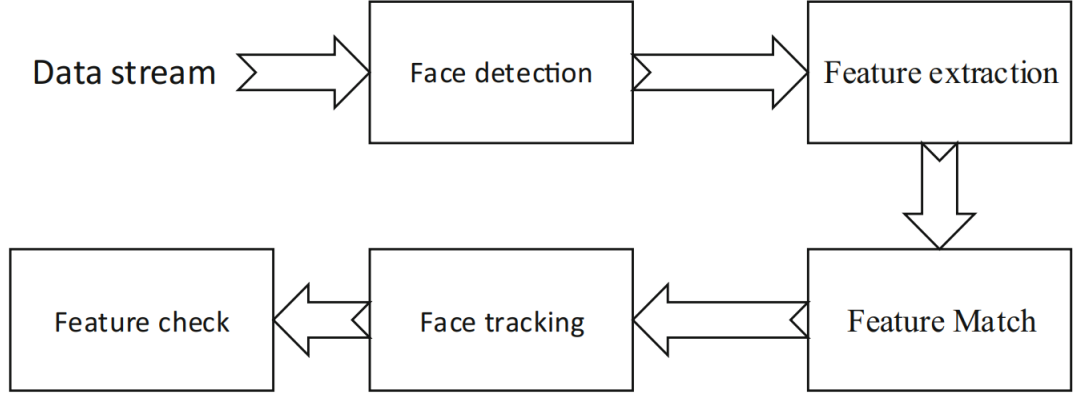


Figure 4.1: System Block Diagram

4.3.1 Dataset Creation and Image Preprocessing

Images of Employee or student are captured using a web cam. Multiple images of single student will be acquired with varied gestures and angles. These images undergo pre-processing. The images are cropped to obtain the Region of Interest (ROI) which will be further used in recognition process. Then these images will be converted from RGB to gray scale images. This is done using OpenCV.

1. Step1: we need to read the image with OpenCV's `imread()` function:
`img = cv2.imread(imagePath)`
This will load the image from the specified file path and return it in the form of a Numpy array.
`img.shape`
Output:
(1000,500,4)
2. Step2: Convert the RGB data of Image into Greyscale To improve computational efficiency, we first need to convert this image to grayscale before performing face detection on it: Code
`gray_image = cv2.cvtColor(img, cv2.COLOR_BGR2_GRAY)` Output:
(1000,500)
This array only has two values since the image is grayscale and no longer has the third color channel.

4.3.2 Face Detection

The idea behind this technique involves using a cascade of classifiers to detect different features in an image. Haar Cascade algorithm needs to be trained to

detect human faces before it can be used for face detection. This is called feature extraction. The haar cascade training data used is an xml file- `haarcascade_frontalface_default`. We are using a file called `haarcascade_frontalface_default.xml`. This classifier is designed specifically for detecting frontal faces in visual input. `face_classifier = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")`. We are using `detectMultiScale` module from OpenCV. This is required to create a rectangle around the faces in an image. It has got three parameters to consider- `scaleFactor`, `minNeighbors`, `minSize`.

4.3.3 Face Recognition and Calculation

Face recognizer used in this system is Local Binary Pattern Histogram. Initially, the list of local binary patterns (LBP) of entire face is obtained. These LBPs are converted into decimal number and then histograms of all those decimal values are made. At the end, one histogram will be formed for each images in the training data. Later, during recognition process histogram of the face to be recognized is calculated and then compared with the already computed histograms and returns the best matched label associated .

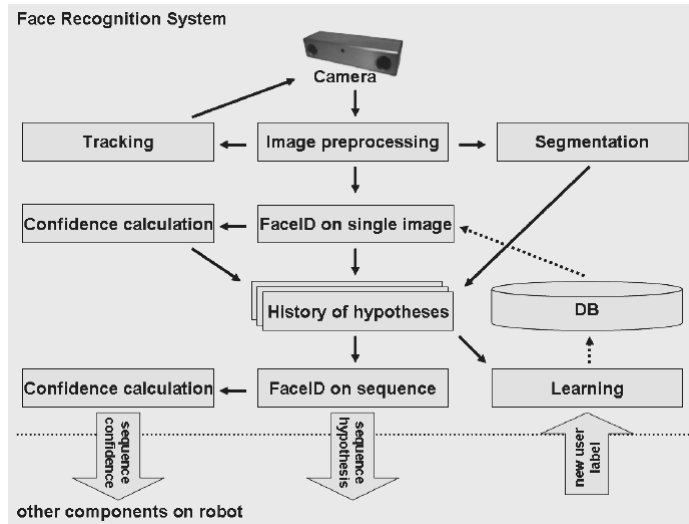


Figure 4.2: System architecture of face recognition system

4.3.4 DataBase

After face recognition process, based on Calculated histogram Databases are updated .The name of people along with address, gender, birthday and time of update is also be stored in the database.

4.3.5 Real-Time Tracking

The triplet loss is particularly effective for training on face recognition tasks. The basic idea is to have triplets of images: an anchor image (A), a positive image (P) of the same person as the anchor, and a negative image (N) of a different person.

The triplet loss function is defined as follows:

$$L(A, P, N) = \max(0, \|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha)$$

Here:

$f(\cdot)$ represents the embedding function (the output of the neural network for a given input).

$\|\cdot\|^2$ denotes the squared Euclidean norm.

α is a margin that ensures the positive pair should be closer by at least this margin than the negative pair.

In other words, the loss encourages the distance between the anchor and positive embeddings to be smaller than the distance between the anchor and negative embeddings by at least the margin α . If this condition is not satisfied, the loss becomes non-zero, and the network is updated to improve the embeddings.

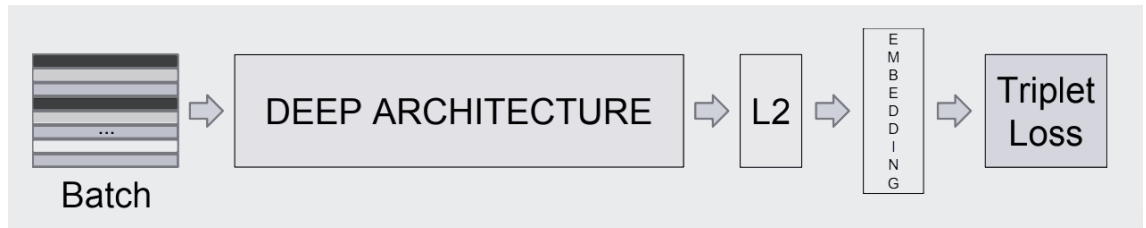


Figure 4.3: FaceNet Architecture

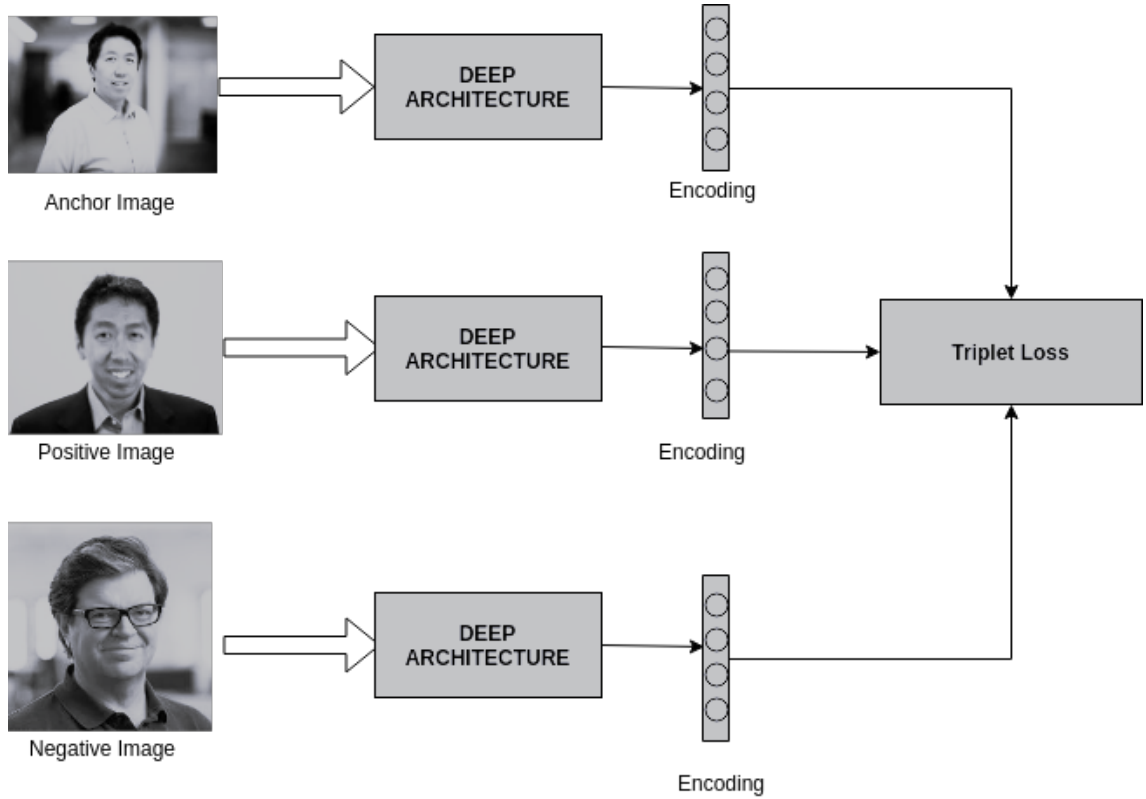


Figure 4.4: Triplet Loss

Chapter 5

Implementation Plan

5.1 Schedule(Gantt-Chart)

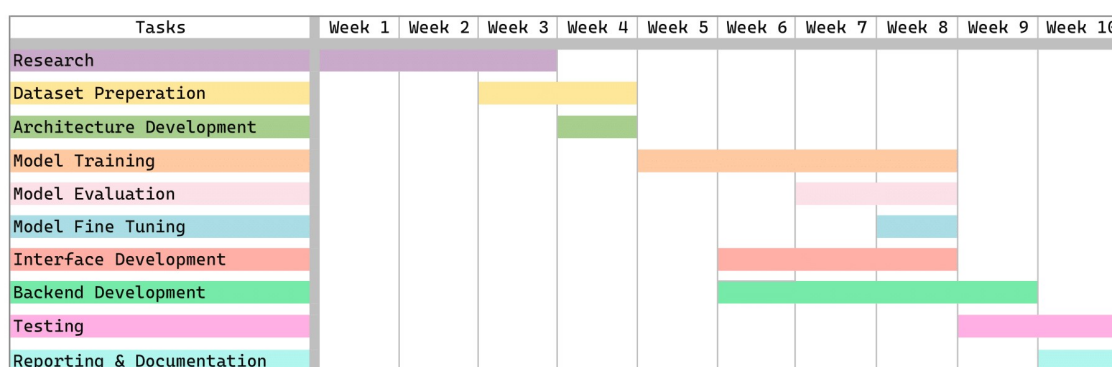


Figure 5.1: Gantt Chart

5.2 SOFTWARE REQUIREMENT

Our Facial Recognition system requires Python, OpenCV, MySQL, TensorFlow, Flask, CNN, JavaScript, Keras which are described below;

5.2.1 Python

Python is a high-level interpreted programming language for general-purpose programming created by Guido van Rossum which was released in 1991. Python has design philosophy that emphasizes code readability, notably using significant white spaces. The programming language features dynamic type system and automatic memory management with support to multiple programming paradigms including Object Oriented Imperative, Functional and Procedural. The programming language has large comprehensive standard library.

5.2.2 OpenCV (Open Source Computer Vision Library)

OpenCV is a popular open-source library designed for computer vision tasks. It provides a wide range of functions and algorithms related to image and video

processing. OpenCV is widely used for tasks like image and video processing, object detection, facial recognition, and more. It is written in C++ but has Python bindings, making it accessible to a broader audience.

5.2.3 MySQL

MySQL is an open-source relational database management system (RDBMS). Its name is a combination of "My", the name of co-founder Michael Widenius's daughter, and "SQL", the abbreviation for Structured Query Language. In our project, we use this database to feed into PHP system for data labeling. The database is created from scraping Nepali Sentiment text in python.

5.2.4 Tensor Flow

TensorFlow is an open-source deep learning framework developed by the Google Brain team. It is widely used for building and training machine learning and deep learning models. It provides Flexible architecture for building various machine learning models. It supports deep learning, including neural networks and convolutional neural networks (CNNs). TensorFlow is used for developing and training deep learning models for tasks such as image classification, object detection, natural language processing, and more.

5.3 FUNCTIONAL REQUIREMENT

5.3.1 Face Detection

Accurate face detection algorithms should be implemented to identify and locate faces in images or video streams.

5.3.2 Face Recognition

Employ robust facial recognition algorithms for accurate identification of registered individuals. The system should provide a confidence score or percentage to indicate the level of certainty in the recognition process.

5.3.3 Database Management

The system should manage a secure database of people data and their corresponding facial features. Allow easy addition, modification, or removal of people from the database.

5.3.4 Real-time Monitoring

Provide real-time monitoring capabilities for administrators.

5.4 NON-FUNCTIONAL REQUIREMENT

These requirements are not needed by the system but are essential for the better performance of sentiment engine. The points below focus on the non-functional requirement of the system.

5.4.1 Performance

5.4.1.1 Response Time

The system should have low latency in recognizing faces and tracking recognized person.

5.4.1.2 Throughput

It should be able to handle a specified number of facial recognition and tracking.

5.4.2 Scalabiity

The system should be scalable to accommodate an increasing number of people.

5.4.3 Reliability

The system should be highly reliable, minimizing the risk of false positives or negatives in facial recognition. It should have mechanisms to handle system failures and recover gracefully.

5.4.4 Availability

The system should have a high level of availability to ensure that it is accessible when needed.

5.4.5 Security

5.4.5.1 Data Security

Facial data should be stored and transmitted securely, following industry best practices.

5.4.5.2 Access Control

Implement robust access control mechanisms to restrict unauthorized access to the system.

Chapter 6

Expected Outcomes

The users can interact with the system using a GUI. This method will have higher accuracy in recognition of multiple faces from a single frame with lower response time.

The best model was found to be with FaceNet having testing accuracy exceed 99%. Other models performed less than this model is due to, FaceNet is a state-of-the-art face recognition system that leverages deep learning techniques, specifically Convolutional Neural Networks (CNNs), to map facial features into a high-dimensional space called an embedding. Other problem is to detect high variability of human faces in terms of shape, size, pose, expression, illumination, occlusion, and makeup. FaceNet uses a triplet loss function during training. The loss is computed based on triplets of images: an anchor image of a face, a positive image of the same face (from a different view or under different lighting conditions), and a negative image of a different face. Other model are MTCNN(Multi-task Cascaded Convolutional Networks) and CNN(Convolutional Neural Network) is known for its efficiency and accuracy in face detection tasks.

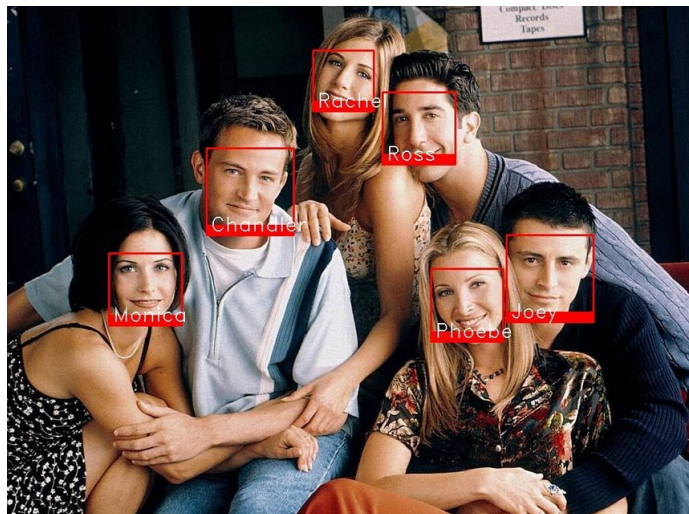


Figure 6.1: face recognition output

However, all above models have high accuracy in facial recognition, for face tracking we can integrate tracking algorithm mention above. we like to go for Correlation Filter Tracking which is best for real time tracking with high performance when combine with MTCNN, FaceNet and CNN facial recognition models.



Figure 6.2: multiPerson-Tracking

6.1 Conclusion

The implementation of a Facial Recognition Along With Real-Time Tracking aligns with technological advancements, offering a more accurate and efficient solution for Facial tracking of person. This proposal lays the foundation for a robust system that addresses the current challenges in security surveillance with very high accuracy of recognition and tracking.

Bibliography

- [1] Muhammet Baykara and Resul Das. Real time face recognition and tracking system. pages 159–163, 11 2013.
- [2] Jun Li, Yaoru Wang, Guokang Fang, and Zhihui Zeng. Real-time detection tracking and recognition algorithm based on multi-target faces. *Multimedia Tools and Applications*, 80(11):17223–17238, August 2020.
- [3] Teh-Lu Liao, Hsin-Chieh Chen, Qing-Huang Song, and Yi-You Hou. Face recognition and real-time tracking system based on convolutional neural network and parallel-cascade pid controller. *Measurement and Control*, 55(7–8):616–630, July 2022.
- [4] Shou-Jen Lin, Chao-Yang Lee, Mei-Hsuan Chao, Chi-Sen Chiou, and Chu-Sing Yang. The study and implementation of real-time face recognition and tracking system. In *2010 International Conference on Machine Learning and Cybernetics*, volume 6, pages 3050–3055, 2010.
- [5] Artak Melkumyan and Katya Mkrtchyan. Real time facial recognition and tracking system using drones. In *2023 IEEE 20th Consumer Communications and Networking Conference (CCNC)*, pages 975–976, 2023.
- [6] Wesley L. Passos, Igor M. Quintanilha, and Gabriel M. Araujo. Real-time deep-learning-based system for facial recognition. *Anais De XXXVI Simpósio Brasileiro De Telecomunicações E Processamento De Sinais*, January 2018.
- [7] K. Bharath S Reddy, Onkar Loke, Shantanu Jani, and Kanchan Dabre. Tracking people in real time video footage using facial recognition. *2018 International Conference on Smart City and Emerging Technology (ICSCET)*, January 2018.